

IES ALBARREGAS DAW DUAL

Proyecto Intermodular

PI · Unidad 2 Tarea 06

Skillin | Manual Técnico de la Aplicación



Constantino Alexopoulos Real

16 de julio de 2026



Manual Técnico de la aplicación

Plataforma web gamificada para la
evaluación y entrenamiento de
competencias profesionales
en empresas

Índice de contenidos

1. Introducción y alcance	04
2. Arquitectura general	04
2.1. Flujo de una petición	04
2.2. Autoload	05
3. Stack tecnológico	05
4. Estructura de directorios	06
5. Instalación y despliegue	07
5.1. Requisitos	07
5.2. Pasos	07
5.3. Cuentas de prueba (seed_passwords.php)	07
6. Configuración	08
6.1. config/config.php	08
6.2. config/database.php	09
6.3. config/mail.php	09
7. Modelo de datos	10
7.1. Diagrama entidad-relación (textual)	10
7.2. Tablas	10
8. Núcleo de la aplicación (app/core)	13
8.1. Router.php	13
8.2. Controller.php (clase base abstracta)	13
8.3. Auth.php	14
8.4. Mailer.php	14
8.5. config/database.php -> clase Database	14
9. Mantenimiento y líneas de ampliación	15
10. Mapa de rutas	16
11. Módulos funcionales	17
11.1. Autenticación	17
11.2. Dashboard	17
11.3. Perfil	17
11.4. Gestión de usuarios / plantilla	18
11.5. Catálogo de juegos	18
11.6. Asignación de juegos	19
11.7. Resultados y “Mi progreso”	19
11.8. Informes	19
11.9. Empresas	20
12. Subida de ficheros	21
13. Los serious games	22
14. Seguridad	23
15. Convenciones de código	24
16. Guías de extensión	25
17. Limitaciones conocidas y mejoras futuras	26

1. Introducción y alcance

Skillin es una aplicación web que permite a una empresa evaluar y entrenar competencias profesionales de su plantilla mediante *serious games* (minijuegos con un objetivo formativo). El personal de RRHH (o un administrador con visión multiempresa) asigna juegos a los trabajadores, estos los completan, y el sistema registra puntuación y tiempo para generar informes de rendimiento.

La aplicación distingue tres roles (trabajador, rrhh, administrador) y gestiona el ciclo completo: alta de empresas y usuarios, catálogo de juegos, asignación, ejecución, resultados e informes.

Este documento cubre exclusivamente el backend/frontend de la aplicación PHP (no incluye el modelo E/R entregado como documento separado en la Tarea 03, aunque el esquema de BD aquí descrito lo implementa fielmente).

2. Arquitectura general

Skillin implementa un patrón MVC hecho a mano, sin ningún framework (no hay *Composer*, ni *Laravel/Symfony*, ni un ORM). Todo el código es PHP 8 “*vanilla*” con acceso a datos vía PDO y sentencias preparadas.

2.1. Flujo de una petición

No existe capa de “Servicio” ni “Repositorio” intermedia: el *Controller* llama directamente a los Modelos. Es una arquitectura simple, apropiada para el tamaño del proyecto.

```
Navegador
| HTTP request a /skillin/public/<ruta>
v
public/.htaccess      → reescribe cualquier ruta no física hacia index.php
v
public/index.php      → Front Controller: carga config, core y modelos,
                        registra las rutas y despacha
v
app/core/Router.php    → busca la ruta que coincide (metodo + patron),
                        instancia el Controller y llama a la acción
v
app/controllers/*.php  → hereda de Controller (app/core/Controller.php):
                        valida sesión/rol (Auth), lee input, llama a
                        uno o varios Modelos, y renderiza una Vista
                        o hace un redirect/JSON
v
app/models/*.php       → una clase por tabla principal; PDO + SQL
                        preparado, sin lógica de presentación
v
app/views/**/*.php    → plantillas PHP con HTML embebido; reciben los
                        datos ya preparados por el controlador vía
                        extract() (Controller::view() / viewOnly())
```

2.2. Autoload

No hay *autoload PSR-4* ni *Composer*. Las clases se cargan explícitamente:

- **public/index.php** hace `require_once` de *Auth*, *Controller*, *Router* y *Mailer* (núcleo), y recorre con `glob()` todos los ficheros de `app/models/*.php` para cargarlos automáticamente.
- Los **controladores** se cargan bajo demanda desde `Router::dispatch()` (`require_once app/controllers/{Controller}.php`) según el nombre indicado en la ruta.

Esto implica que cualquier modelo nuevo se detecta automáticamente (basta con crear el fichero en `app/models/`), pero un controlador nuevo solo funciona si se registra su ruta en `public/index.php`.

3. Stack tecnológico

Capa	Tecnología
Lenguaje backend	PHP 8.1+ (tipado de propiedades/parámetros, match, str_contains, etc.)
Base de datos	MySQL 5.7+ / MariaDB 10.4+ (motor InnoDB, utf8mb4)
Acceso a datos	PDO nativo, sentencias preparadas, PDO::ATTR_EMULATE_PREPARES = false
Servidor web	Apache + mod_rewrite (XAMPP en desarrollo)
Frontend	HTML + CSS propio (public/assets/css/style.css, sin framework CSS) + JavaScript vanilla embebido en las vistas de los juegos
Gráficos	Chart.js (cargado por CDN únicamente en rrhh/informes.php)
Envío de email	Cliente SMTP propio sobre sockets (app/core/Mailer.php), sin librerías externas (no hay Composer en el proyecto)
Sesiones	Sesiones nativas de PHP (session_start(), cookies de sesión)

4. Estructura de directorios

skillin/	
-- app/	
-- controllers/	Un controlador por recurso
-- AuthController.php	Login, registro, logout, recuperación
-- DashboardController.php	Panel segun rol
-- PerfilController.php	Datos propios, avatar, contraseña
-- UsuarioController.php	CRUD de usuarios (RRHH/Administrador)
-- JuegoController.php	Catálogo + ejecución de juegos
-- AsignacionController.php	Asignación de juegos a trabajadores
-- InformeController.php	Informes y analítica
`-- EmpresaController.php	Alta de empresas (solo administrador)
-- core/	
-- Router.php	Enrutador ligero (patron {param})
-- Controller.php	Clase base: vistas, redirects, CSRF, auth/rol
-- Auth.php	Sesión de usuario (login/logout/rol)
`-- Mailer.php	Cliente SMTP mínimo (STARTTLS/SSL + AUTH LOGIN)
-- models/	Una clase por tabla (PDO + SQL preparado)
`-- Usuario, Empresa, Juego,	
AsignacionJuego, Resultado,	
Informe, PasswordReset	
`-- views/	
-- layouts/	header.php (sidebar + <head>) y footer.php
-- auth/	login, register, recuperar, reset
-- trabajador/	dashboard, juegos ("Mis juegos"), progreso
-- rrhh/	dashboard, usuarios, juegos, asignaciones, informes
-- admin/	empresas
-- perfil/	index (datos + avatar + contraseña)
-- juegos/	quiz.php, memoria.php, reaccion.php
`-- errors/	403.php, 404.php
-- config/	
-- config.php	BASE_URL, zona horaria, arranque de sesión
-- database.php	Clase Database (Singleton PDO)
`-- mail.php	Credenciales SMTP
-- database/	
-- skillin.sql	Script completo de creación + datos de prueba
`-- seed_passwords.php	Genera los hashes bcrypt reales de las cuentas demo
-- public/	DocumentRoot - único punto de entrada HTTP
-- index.php	Front Controller
-- .htaccess	Reescritura hacia index.php
-- assets/	
-- css/style.css	Estilos (paleta Skillin, variables CSS)
`-- img/	Logos
`-- uploads/	
-- avatars/	Fotos de perfil (u{id_usuario}.ext)
`-- juegos/	Imágenes de cabecera (j{id_juego}.ext)
`-- README.md	Guía rápida de instalación

5. Instalación y despliegue

5.1. Requisitos

- PHP 8.1+ con extensión pdo_mysql.
- MySQL 5.7+ / MariaDB 10.4+.
- Apache con mod_rewrite y AllowOverride All (o el servidor embebido php -S para pruebas rápidas).

5.2. Pasos

- **1. Copiar el proyecto** dentro de htdocs/ (XAMPP) o apuntar el DocumentRoot/Alias a la carpeta public/.
- **2. Importar la base:** `mysql -u root -p < database/skillindb.sql`.
Editar el script o config/database.php para que apunten a la misma base de datos.
- **3. Regenerar contraseñas reales** de las cuentas de prueba con `php database/seed_passwords.php` (los hashes del .sql son ilustrativos).
- **4. Configurar la conexión** en config/database.php (host, nombre de BD, usuario, contraseña).
- **5. Configurar BASE_URL** en config/config.php según la ruta de despliegue (p. ej. /skillin/public, o " si public/ es la raíz del dominio).
- **6. Configurar el envío de email** (opcional) en config/mail.php para que funcione "Recuperar contraseña".
- **7. Acceder** a <http://localhost/skillin/public/> (o la URL configurada).

5.3. Cuentas de prueba (tras seed_passwords.php)

Rol	Email	Contraseña
Administrador	admin@skillin.com	1234 (ajustar en <i>seed_passwords.php</i>)
RRHH	rrhh@albarregas.es	1234
Trabajador	ana.garcia@albarregas.es	1234
Trabajador	carlos.ruiz@albarregas.es	1234

El fichero database/seed_passwords.php es la fuente de verdad de qué cuentas y contraseñas se regeneran; editálo si se añaden nuevas cuentas demo.

6. Configuración

6.1. config/config.php

```
define('BASE_URL', '/skillin/public');// prefijo de todas las URLs generadas
define('APP_NAME', 'Skillin');
define('SESSION_LIFETIME', 7200); // 2 horas
date_default_timezone_set('Europe/Madrid');
session_set_cookie_params(SESSION_LIFETIME);
session_start()
```

Es el primer fichero que se carga (*public/index.php*); arranca la sesión PHP nativa que usa Auth para guardar el usuario logueado.

6.2. config/database.php

Clase Database con patrón Singleton sobre PDO:

```
private const HOST      = 'localhost';
private const DBNAME    = 'skillindb';
private const USER     = 'root';
private const PASS      = '';
```

Opciones PDO relevantes:

- **PDO::ATTR_ERRMODE = PDO::ERRMODE_EXCEPTION** -> cualquier error SQL lanza PDOException (no hay manejo global de excepciones: se traduce en un error 500 con el mensaje de PHP, salvo en el propio constructor de Database, que captura el fallo de conexión y lo muestra con die()).
- **PDO::ATTR_EMULATE_PREPARES = false** -> los prepares son reales (los ejecuta MySQL, no PDO). Esto tiene una implicación importante: no se puede repetir el mismo marcador con nombre dos veces en la misma consulta (p. ej. WHERE a LIKE :x OR b LIKE :x falla con SQLSTATE[HY093]: Invalid parameter number); hay que usar marcadores distintos por cada aparición aunque el valor sea el mismo (ver Usuario: listarPorEmpresa() como ejemplo del patrón correcto).

6.3. config/mail.php

Configuración del cliente SMTP propio (*app/core/Mailer.php*), usada por la recuperación de contraseña:

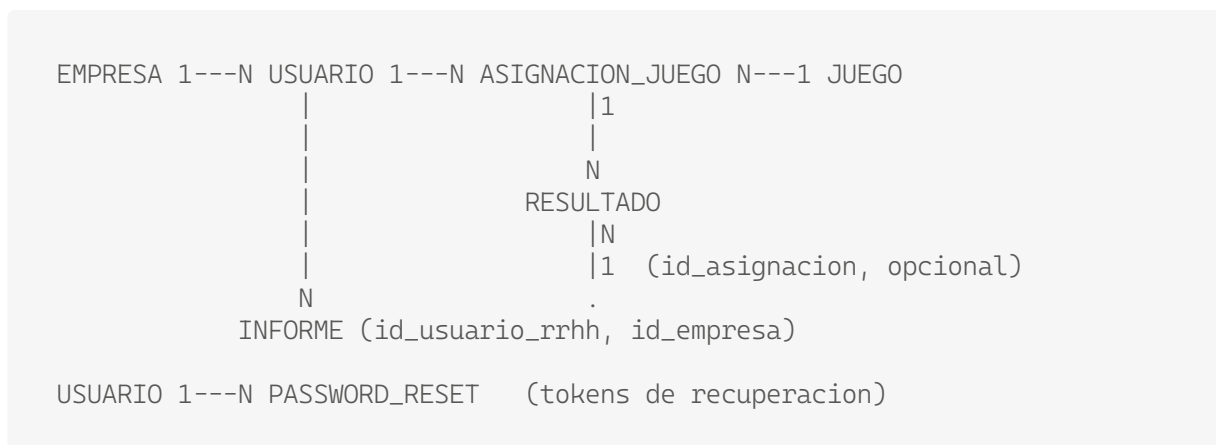
```
return [  
    'host'          ⇒ 'smtp.example.com',  
    'port'          ⇒ 587,  
    'encryption'    ⇒ 'tls', // 'tls' (STARTTLS) o 'ssl'  
    'username'      ⇒ '',  
    'password'      ⇒ '',  
    'from_email'    ⇒ 'no-reply@skillin.com',  
    'from_name'     ⇒ 'Skillin',  
];
```

Mientras host mantenga el valor de ejemplo (smtp.example.com), Mailer::enviar() no intenta conectar: registra un aviso en el log de PHP/Apache y devuelve false sin romper el flujo (el token de recuperación se genera igualmente en BD, solo falla el envío del correo).

7. Modelo de datos

Motor InnoDB, charset utf8mb4. Todas las claves foráneas están declaradas explícitamente con ON DELETE/ON UPDATE.

7.1. Diagrama entidad-relación (textual)



7.2. Tablas

empresa

Campo	Tipo	Descripción
id_empresa	INT UNSIGNED PK AI	
nombre	VARCHAR(150) NOT NULL	
sector	VARCHAR(100) NULL	
fecha_registro	DATETIME	DEFAULT CURRENT_TIMESTAMP

usuario

Campo	Tipo	Descripción
id_usuario	INT UNSIGNED (PK, AI)	Identificador único.
nombre / apellidos	VARCHAR(150)	Datos identificativos.
email	VARCHAR(100) UNIQUE	Usado como credencial de acceso (RF1).
contrasena	VARCHAR(255)	Hash bcrypt (password_hash), nunca texto plano (RNF1).
rol	ENUM('trabajador','rrhh')	Determina la interfaz y permisos (RF2).
departamento	VARCHAR(100)	Campo informativo, opcional.
activo	TINYINT(1)	Permite desactivar sin borrado físico (RF6).
id_empresa	INT UNSIGNED (FK → empresa)	Empresa a la que pertenece.
foto	VARCHAR(255) NULL	nombre de fichero en public/uploads/avatars/
fecha_alta	DATETIME	DEFAULT CURRENT_TIMESTAMP

Índices: *idx_usuario_empresa*, *idx_usuario_rol*.

Semántica del rol:

- **trabajador:** ejecuta los juegos que le asignan.
- **rrhh:** gestiona plantilla, catálogo, asignaciones e informes, acotado a su propia empresa (id_empresa de la sesión).
- **administrador:** igual que rrhh pero sin esa restricción - puede elegir cualquier empresa, y da de alta usuarios y empresas nuevas.

password_reset

Campo	Tipo	Descripción
id_reset	INT UNSIGNED PK AI	
id_usuario	INT UNSIGNED NOT NULL, FK -> usuario	ON DELETE CASCADE
token_hash	VARCHAR(255) NOT NULL	hash SHA-256 del token, nunca el token en claro
expira_en	DATETIME NOT NULL	vida útil: 1 hora desde la generación
usado	TINYINT(1) DEFAULT 0	se marca a 1 tras consumirse (o al pedir uno nuevo)
creado_en	DATETIME	DEFAULT CURRENT_TIMESTAMP

Índices: idx_reset_usuario, idx_reset_token.

juego

Campo	Tipo	Descripción
id_juego	INT UNSIGNED (PK, AI)	Identificador único.
titulo	VARCHAR(150) NOT NULL	Información mostrada al trabajador.
descripcion	VARCHAR(150) NOT NULL	
tipo_competencia	VARCHAR(100) NULL	Competencia que entrena (memoria, atención...).
dificultad	ENUM('facil','media','dificil')	Nivel de dificultad.
slug	VARCHAR(50) UNIQUE NOT NULL	Identifica el motor JS a cargar: quiz, memoria o reaccion.
imagen	VARCHAR(255) NULL	nombre de fichero en public/uploads/juegos/
activo	TINYINT(1)	solo los juegos activos son asignables (Juego::all(true))

asignacion_juego (entidad intermedia N:M usuario <-> juego)

Campo	Tipo	Descripción
id_asignacion	INT UNSIGNED (PK, AI)	Identificador único.
id_usuario	INT UNSIGNED (FK → usuario)	ON DELETE CASCADE. Trabajador al que se asigna el juego.
id_juego	INT UNSIGNED (FK → juego)	ON DELETE CASCADE Juego asignado.
fecha_asignacion	DATETIME	opcional
fecha_limite	DATE NULL	DATETIME
estado	ENUM('pendiente','en_progreso','completado','caducado')	Ciclo de vida de la asignación. Por defecto pendiente
asignado_por	INT UNSIGNED (FK → usuario)	quién hizo la asignación; ON DELETE SET NULL

resultado (histórico de partidas)

Campo	Tipo	Descripción
id_resultado	INT UNSIGNED (PK, AI)	
puntuacion	INT UNSIGNED	
tiempo_empleado	INT UNSIGNED	segundos
fecha_realizacion	DATETIME	
id_usuario, id_juego	FK, ON DELETE CASCADE	
id_asignacion	FK -> asignacion_juego, nullable	ON DELETE SET NULL; permite resultados sin asignación explícita

informe (metadatos de informes generados)

Campo	Tipo	Descripción
id_informe	INT UNSIGNED PK AI	
fecha_generacion	DATETIME	
tipo	VARCHAR(50)	rendimiento participacion competencias (texto libre)
observaciones	TEXT NULL	
id_usuario_rrhh	FK -> usuario	quién lo generó
id_empresa	FK -> empresa	

El informe no almacena los datos calculados: solo queda constancia de que se generó uno (tipo, fecha, autor, observaciones). Los números mostrados en pantalla/CSV se recalculan siempre en caliente a partir de resultado (ver Resultado::rendimientoPorJuego() / estadisticasEmpresa()), evitando duplicar información.

8. Núcleo de la aplicación (*app/core*)

8.1. Router.php

Enrutador mínimo basado en expresiones regulares:

- `get(path, 'Controller@accion')`, `post(...)`, `any(...)` registran rutas.
- Los segmentos `{nombre}` del patrón se traducen a `([^\/]*)` y se pasan como argumentos posicionales al método del controlador (por eso `JuegoController::jugar(string $idAsignacion)` recibe el id como parámetro).
- `dispatch()` obtiene la URL, le quita el prefijo `BASE_URL`, busca la primera ruta cuyo método y patrón coincidan, instancia el controlador (`require_once app/controllers/{Controller}.php`) y llama al método con `call_user_func_array`.
- Si no hay coincidencia -> `notFound()` (renderiza `errors/404.php` con 404).
- Si el controlador o el método no existen -> también 404 (evita errores fatales por rutas mal registradas).

8.2. Controller.php

Todos los controladores heredan de aquí. Métodos clave:

Método	Tipo
<code>view(\$vista, \$datos)</code>	Renderiza <code>header.php</code> + la vista + <code>footer.php</code> , con <code>extract(\$datos)</code> previo
<code>viewOnly(\$vista, \$datos)</code>	Igual pero sin el layout (login, registro, errores, recuperación)
<code>redirect(\$url)</code>	Cabecera Location: <code>BASE_URL/\$url</code> + exit
<code>json(\$datos, \$status)</code>	Respuesta JSON (usada por el guardado de resultados vía fetch)
<code>requireAuth()</code>	Redirige a /login si no hay sesión
<code>requireRole(\$rol)</code>	Exige sesión + rol exacto; administrador pasa cualquier <code>requireRole()</code> . Si falla, 403
<code>empresaSeleccionada(\$user)</code>	Empresa "efectiva": la propia, salvo administrador (lee <code>?empresa=</code>)
<code>input(\$clave, \$default)</code>	Lee de <code>\$_POST</code> y si no está de <code>\$_GET</code>
<code>csrfToken()</code> / <code>checkCsrf()</code>	Genera/valida el token CSRF guardado en <code>\$_SESSION</code>

Importante sobre `requireRole()`: como el administrador satisface cualquier llamada a `requireRole('rrhh')`, todas las pantallas de gestión (`UsuarioController`, `JuegoController`, `AsignacionController`, `InformeController`) están accesibles para el administrador sin tener que duplicar código de permisos en cada controlador —el único sitio donde se diferencia explícitamente el alcance es en `empresaSeleccionada()` y en los métodos de los modelos que reciben `id_empresa` como parámetro. Las acciones exclusivas del administrador (crear empresas, ver/gestionar cuentas RRHH y administrador de otras empresas) usan `requireRole('administrador')` directamente (p. ej. `EmpresaController`).

8.3. Auth.php

Envoltorio estático sobre `$_SESSION['user']`:

- `attempt($email, $password)`: busca por email, comprueba activo y `password_verify()`; si es correcto llama a `login()`.
- `login($usuarioRow)`: hace `session_regenerate_id(true)` (previene session fixation) y guarda en sesión un subconjunto de columnas (`id_usuario`, nombre, apellidos, email, rol, departamento, foto, `id_empresa`) — nunca la contraseña.
- `logout()`: vacía `$_SESSION`, expira la cookie de sesión y `session_destroy()`.
- `check()`, `user()`, `id()`: helpers de lectura.
- `isRrhh()`, `isTrabajador()`, `isAdministrador()`: comprobación exacta de un rol.
- `hasAnyRole(...$roles)`: comprobación de pertenencia a una lista (usado en las vistas para mostrar/ocultar menús).

8.4. Mailer.php

Cliente SMTP hecho a mano sobre `stream_socket_client` (no usa `mail()` de PHP ni ninguna librería como PHPMailer, porque el proyecto no tiene gestor de dependencias). Soporta:

- Conexión en claro + STARTTLS, o conexión `ssl://` directa, según `encryption` en `config/mail.php`.
- AUTH LOGIN (usuario/contraseña en base64) si `username` no está vacío.
- Validación del código de respuesta SMTP esperado en cada paso (EHLO, MAIL FROM, RCPT TO, DATA...); si algo falla lanza una excepción interna que se captura y se traduce en `error_log()` + `return false`.
- Cabeceras From/To/Subject codificadas en `=?UTF-8?B?...?=` para admitir acentos.
- Escapado de líneas que empiecen por `."` en el cuerpo (regla RFC 5321 para no truncar el mensaje).

`enviar(paraEmail, paraNombre, asunto, cuerpoHtml)`: bool es el único método público. Se usa exclusivamente desde `AuthController::recuperar()`.

8.5. config/database.php -> clase Database

Ver 6.2. Es el único punto de conexión PDO; todos los modelos llaman a `Database::getConnection()` en su constructor.

9. Roles y control de acceso

Capacidad	Trabajador	RRHH	Administrador
Ver "Mis juegos" / "Mi progreso"	●	⊖	⊖
Jugar y registrar resultados	●	⊖	⊖
Editar su perfil, avatar y contraseña	●	●	●
Gestionar plantilla (/rrhh/usuarios)	⊖	●	●
Gestionar catálogo de juegos	⊖	●	●
Asignar juegos	⊖	●	●
Ver informes / exportar CSV	⊖	●	●
Elegir empresa vía selector	⊖	⊖	●
Dar de alta empresas nuevas	⊖	⊖	●
Crear cuentas con rol administrador	⊖	⊖	●

Reglas de implementación relevantes:

- El guardián central es `Controller::requireRole()`: un solo punto de código decide que administrador pasa cualquier comprobación de rol.
- El alcance por empresa se resuelve con `Controller::empresaSeleccionada()`: para RRHH siempre devuelve su propia empresa (ignora cualquier intento de manipular `?empresa=` por URL), para administrador respeta el parámetro.
- Autorización a nivel de fila en `UsuarioController` (método privado `puedeGestionar()`): un RRHH solo puede editar/activar/eliminar cuentas trabajador de su propia empresa (comprobado contra la fila real en BD, no solo contra el filtro de listado); el administrador puede gestionar cualquier cuenta. Esto evita que un RRHH manipule el id en la URL para tocar usuarios de otra empresa.
- Las vistas ocultan enlaces de menú según rol (`app/views/layouts/header.php`, `Auth::hasAnyRole()`/`Auth::isAdmin()`), pero la autorización real vive en el controlador, no en la vista - ocultar un enlace es solo una ayuda de UX.

10. Mapa de rutas

Definidas en `public/index.php`, en el orden en que las evalúa el Router (la primera coincidencia gana).

Método	Ruta	Controlador@acción	Notas
GET	/ , login	AuthController@loginForm	-
POST	login	AuthController@login	-
GET	registro	AuthController@registerForm	-
POST	registro	AuthController@register	siempre crea rol trabajador
GET	logout	AuthController@logout	-
GET/ POST	recuperar	AuthController@recuperarForm / recuperar	solicitud de recuperación
GET/ POST	recuperar/reset/{token}	AuthController@resetForm / reset	fijar nueva contraseña
GET	dashboard	DashboardController@index	panel según rol
GET	juegos	JuegoController@catalogo	"Mis juegos" (trabajador)
GET	progreso	JuegoController@progreso	"Mi progreso" (trabajador)
GET	juegos/jugar/{id}	JuegoController@jugar	{id} = id de la asignación
POST	juegos/resultado	JuegoController@guardarResultado	endpoint AJAX (JSON)
GET	perfil	PerfilController@index	-
POST	perfil/actualizar	PerfilController@actualizar	-
POST	perfil/foto	PerfilController@actualizarFoto	-
POST	perfil/password	PerfilController@cambiarPassword	-
GET	rrhh/usuarios	UsuarioController@index	-
POST	rrhh/usuarios/crear	UsuarioController@crear	-
POST	rrhh/usuarios/{id}/actualizar	UsuarioController@actualizar	-
POST	rrhh/usuarios/{id}/toggle	UsuarioController@toggleActivo	-
POST	rrhh/usuarios/{id}/eliminar	UsuarioController@eliminar	-
GET	rrhh/juegos	JuegoController@gestion	catálogo (gestión)
POST	rrhh/juegos/crear	JuegoController@crear	admite multipart/form-data (imagen)
POST	rrhh/juegos/{id}/actualizar	JuegoController@actualizar	-
POST	rrhh/juegos/{id}/imagen	JuegoController@actualizarImagen	sube/reemplaza la imagen
POST	rrhh/juegos/{id}/eliminar	JuegoController@eliminar	-
GET	rrhh/asignaciones	AsignacionController@index	-
POST	rrhh/asignaciones/asignar	AsignacionController@asignar	individual o múltiple
POST	rrhh/asignaciones/{id}/eliminar	AsignacionController@eliminar	-
GET	rrhh/informes	InformeController@index	-
POST	rrhh/informes/generar	InformeController@generar	-
GET	rrhh/informes/exportar/{id}	InformeController@exportarCsv	{id} = id de juego; descarga CSV
GET	admin/empresas	EmpresaController@index	solo administrador
POST	admin/empresas/crear	EmpresaController@crear	solo administrador

Nota de nomenclatura: el prefijo rrhh/ es histórico - bajo él conviven pantallas usadas tanto por rrhh como por administrador (la autorización real la decide requireRole(), no el prefijo de la URL). Solo lo exclusivo del administrador (empresas) usa el prefijo admin/.

11. Módulos funcionales

11.1 Autenticación (login, registro y recuperación de contraseña)

Controlador: AuthController. Modelos: Usuario, Empresa, PasswordReset. Vistas: auth/login.php, auth/register.php, auth/recuperar.php, auth/reset.php.

- **Login:** formulario simple email + contraseña; Auth::attempt() verifica cuenta activa y hash bcrypt.
- **Registro público:** cualquier visitante puede crear una cuenta trabajador eligiendo su empresa de una lista (Empresa::all()). Las cuentas rrhh/administrador no son autoservicio: se crean desde el panel de gestión por alguien que ya tiene ese rol.

Recuperación de contraseña (flujo completo con token de un solo uso):

1. El usuario pide recuperar desde /recuperar con su email.
2. AuthController::recuperar() genera un token aleatorio de 32 bytes (bin2hex(random_bytes(32))), invalida cualquier token pendiente anterior del usuario (PasswordReset::invalidarPendientes()) y guarda el hash SHA-256 del token con expiración a 1 hora (PasswordReset::crear()).
3. Se envía un email con el enlace /recuperar/reset/{token} vía Mailer.
4. Por seguridad, el mensaje mostrado es siempre el mismo exista o no la cuenta con ese email (evita enumeración de usuarios registrados).
5. resetForm()/reset() validan el token (hash + no caducado + no usado) antes de permitir fijar una contraseña nueva; al usarlo, se marca usado = 1 (no reutilizable).

11.2 Dashboard

Controlador: DashboardController. Una única acción index() que se ramifica según rol:

- **administrador y rrhh** -> método privado rrhh(): reutiliza la misma vista (rrhh/dashboard.php) parametrizada por empresaSeleccionada(); el administrador ve además el selector de empresa en la cabecera.
- **trabajador** -> método privado trabajador(): sus asignaciones pendientes, último resultado y contador de partidas.

11.3 Perfil (PerfilController)

Accesible por cualquier rol autenticado (/perfil):

- **Datos personales:** nombre, apellidos, email, departamento (Usuario::update()); tras guardar, se refresca la sesión con Auth::login() para que el nombre/rol mostrados en el sidebar estén al día.

- **Empresa:** se muestra (vía `Usuario::findConEmpresa()`, que hace JOIN con empresa) pero no es editable desde aquí — se fija al dar de alta el usuario.
- **Avatar (`actualizarFoto()`):** sube una imagen (jpg/png/webp, máx. 2 MB), validada por contenido real (`getimagesize()`), no por extensión ni MIME declarado por el navegador) y guardada como `u{id_usuario}.{ext}` en `public/uploads/avatars/`; borra el fichero anterior si cambia de extensión.
- **Contraseña (`cambiarPassword()`):** exige la contraseña actual (`password_verify`), mínimo 8 caracteres y confirmación.

11.4 Gestión de usuarios / plantilla (`UserController`)

Ruta base `/rrhh/usuarios`. Accesible por `rrhh` y administrador (`requireRole('rrhh')`, con el administrador pasando por la excepción ya descrita).

- **Listado (`index()`):** para `RRHH`, `Usuario::listarTrabajadoresPorEmpresa()` (solo rol trabajador de su empresa); para administrador, `Usuario::listarPorEmpresa()` (todas las cuentas, cualquier rol, de la empresa seleccionada) — la vista añade una columna “Rol” solo para el administrador.
- **Alta (`crear()`):** además de los datos del usuario, el administrador puede elegir la empresa destino de un desplegable, o dar de alta una empresa nueva sobre la marcha rellenando nombre (+ sector opcional) en el mismo formulario — se crea primero la empresa (`Empresa::create()`), comprobando que el nombre no exista) y el usuario queda asignado a ella. El rol asignable depende de quién crea: `RRHH` solo puede crear trabajador/`rrhh`; el administrador puede además crear administrador.
- **Actualizar / activar-desactivar / eliminar:** cada acción carga primero la fila objetivo y comprueba `puedeGestionar($actor, $objetivo)` antes de tocar nada (ver §9).

El borrado de una cuenta no es físico por defecto en el flujo normal de uso: se prioriza `toggleActivo()` (activar/desactivar) para no perder el histórico de resultados; `eliminar()` sí borra la fila (y en cascada sus asignaciones/resultados por las FK ON DELETE CASCADE).

11.5 Catálogo de juegos (`JuegoController`, parte de gestión)

Ruta base `/rrhh/juegos`. CRUD del catálogo global de serious games (no está filtrado por empresa: el catálogo es compartido por toda la plataforma).

- **Alta/edición:** título, descripción, tipo de competencia, dificultad, slug (fijo a los tres motores existentes: `quiz/memoria/reaccion`) y estado activo/inactivo.
- **Imagen de cabecera:** subida opcional al crear, o mediante un control dedicado por fila en la tabla de gestión (`actualizarImagen()`); la interacción de subida está pensada para no depender del `<input type="file">` nativo: la propia miniatura (o el marcador “+ Subir” si no hay imagen) actúa como disparador clicable de un input oculto, y al elegir el archivo el formulario se envía automáticamente (`onchange="this.form.submit()"`).
- **Validación de imagen** idéntica a la de avatares (contenido real vía `getimagesize()`, límite 2 MB, guardado como `j{id_juego}.{ext}`).

11.6 Asignación de juegos (AsignacionController)

Ruta base /rrhh/asignaciones.

- Alta (asignar()): checklist de trabajadores + selector de juego + fecha límite opcional; permite asignación individual o por grupo en una sola petición (AsignacionJuego::asignarMultiple()).

Regla de negocio - no duplicar asignaciones activas:

Antes de insertar, por cada trabajador seleccionado se comprueba AsignacionJuego::tienePendiente(\$idUser, \$idJuego), que es true si existe alguna fila de ese par (usuario, juego) en estado distinto de completado (pendiente, en_progreso o caducado). Si es así, ese trabajador se omite silenciosamente de la nueva asignación (no se crea una fila duplicada) y se informa al final cuántos se omitieron (?aviso=omitidos&n=X). Si el trabajador solo tiene instancias completado de ese juego (o ninguna), sí se le puede volver a asignar — se inserta una nueva fila independiente, dejando intacto el histórico de la anterior.

Listado y eliminación: AsignacionJuego::listarPorEmpresa() trae también el nombre del trabajador y el título del juego mediante JOIN.

11.7 Resultados y “Mi progreso”

No tiene controlador propio: los resultados se registran desde JuegoController::guardarResultado() (ver §13) y se consultan desde:

- JuegoController::progreso() -> Resultado::historialPorUsuario() (vista del propio trabajador).
- DashboardController/InformeController -> Resultado::estadisticasEmpresa() y rendimientoPorJuego() (agregados para RRHH/administrador).

11.8 Informes (InformeController)

Ruta base /rrhh/informes.

- **Rendimiento por juego:** selector de juego -> tabla + gráfico de barras (Chart.js) con la puntuación de cada trabajador (Resultado::rendimientoPorJuego()).
- **Estadísticas generales:** media de puntuación y nº de partidas por juego (Resultado::estadisticasEmpresa()), en un segundo gráfico.
- **Generar informe formal (generar()):** guarda solo metadatos (tipo, observaciones, autor, empresa, fecha) en la tabla informe — no fotografía los datos, que siempre se recalculan al vuelo (ver nota en §7.2).
- **Exportar CSV (exportarCsv()):** mismo cálculo que “rendimiento por juego”, volcado a un fichero .csv (Content-Disposition: attachment).

11.9 Empresas (EmpresaController)

Ruta base `/admin/empresas`, exclusiva del rol administrador (`requireRole('administrador')`, sin excepción para RRHH).

- Listado de todas las empresas registradas.
- Alta de empresa nueva (nombre + sector opcional), con comprobación de nombre duplicado (`Empresa::nombreExists()`).
- Esta misma lógica de alta está también embebida en el formulario de creación de usuarios (§11.4), para no obligar al administrador a cambiar de pantalla al dar de alta un usuario en una empresa que todavía no existe.

12. Subida de ficheros

Dos flujos análogos comparten el mismo patrón de seguridad (avatares de perfil y portadas de juego):

1. **Límite de tamaño** comprobado antes de tocar el disco (2 MB).
2. **Validación por contenido**, no por metadatos: se usa `getimagesize($tmp_name)` para obtener el MIME real del fichero subido; no se confía en `$_FILES[...]['type']` (cabecera enviada por el navegador, falsificable) ni en la extensión del nombre original. Solo se aceptan `image/jpeg`, `image/png` y `image/webp`.
3. **Nombre de fichero generado por el servidor**, nunca a partir del nombre original del usuario: `u{id_usuario}.{ext}` para avatares, `j{id_juego}.{ext}` para juegos. Esto evita path traversal y colisiones, y de paso hace que cada entidad tenga como máximo un fichero de imagen (una subida nueva sustituye a la anterior).
4. **Limpieza del fichero anterior** si la extensión cambia (por ejemplo, de `.jpg` a `.png`), para no dejar basura huérfana en `public/uploads/`.
5. **Las carpetas** `public/uploads/avatars/` y `public/uploads/juegos/` contienen un `index.php` que devuelve 403 - evita que, si el listado de directorio estuviera mal configurado en el servidor, se pudiera listar el contenido de la carpeta.

Las imágenes servidas desde `public/uploads/` se sirven directamente por Apache como estáticos (no pasan por el Router/`index.php`); por eso el nombre de fichero controlado por el servidor es la única barrera relevante contra manipulación - no hay comprobación de autorización al leer una imagen (son públicas por diseño, como cualquier avatar o portada de juego).

13. Los serious games

Los tres juegos (quiz, memoria, reaccion) siguen el mismo patrón:

1. JuegoController::jugar(\$idAsignacion) valida que la asignación exista y pertenezca al usuario logueado (si no, 404), marca la asignación como en_progreso y renderiza la vista correspondiente según juego.slug (match en PHP).
2. Toda la mecánica del juego se ejecuta en el navegador (JavaScript vanilla embebido en la vista, sin dependencias): preguntas/tablero/estímulos están hardcodeados en el <script> de cada vista (*app/views/juegos/{quiz,memoria,reaccion}.php*), se calcula la puntuación y el tiempo transcurrido en el cliente.
3. Al terminar, la vista hace un fetch() POST a */juegos/resultado* con id_juego, id_asignacion, puntuacion, tiempo y el token CSRF.
4. JuegoController::guardarResultado() (responde JSON) inserta la fila en resultado (Resultado::registrar()) y, si venía ligado a una asignación, la marca completado (AsignacionJuego::marcarEstado()).

Para añadir un cuarto juego, hacen falta cuatro cosas:

- (a) una fila en juego con un slug nuevo,
- (b) una vista *app/views/juegos/{slug}.php* con su propia mecánica JS que reutilice el mismo contrato fetch() hacia */juegos/resultado*,
- (c) añadir el case correspondiente en el match() de JuegoController::jugar(), y
- (d) opcionalmente añadir el slug al <select> de *rrhh/juegos.php*.

14. Seguridad

Resumen transversal (cada mecanismo está detallado en su sección correspondiente):

- **Contraseñas:** `password_hash()` (bcrypt) al crear/cambiar; `password_verify()` al comprobar. Nunca se guarda ni se compara en claro.
- **SQL:** sentencias preparadas PDO en el 100% de las consultas de los modelos; sin concatenación de input de usuario en SQL. *Prepares* reales (EMULATE_PREPARES = false), lo que exige marcadores distintos si un mismo valor se repite en la consulta (§6.2).
- **CSRF:** todo formulario POST incluye `csrf_token`; `Controller::checkCsrf()` lo valida con `hash_equals()` contra el guardado en sesión antes de procesar cualquier acción con efectos.
- **Sesión:** `session_regenerate_id(true)` en cada login (mitiga session fixation); cookie de sesión con vida limitada (SESSION_LIFETIME); `logout()` limpia y destruye la sesión y expira la cookie.
- **Autorización en profundidad:** verificación de rol (`requireRole`) y de pertenencia (`puedeGestionar()` en `UsuarioController`, filtro por `id_empresa` en el resto de modelos) - no basta con el rol, también se comprueba que el recurso objetivo pertenece al ámbito del actor.
- **Recuperación de contraseña:** token de un solo uso, hash SHA-256 en BD (nunca el token en claro), expiración de 1 hora, mensaje de respuesta uniforme para no revelar qué correos están registrados, invalidación de tokens anteriores al pedir uno nuevo.
- **Subida de ficheros:** validación por contenido real, no por metadatos del cliente; nombre de fichero controlado por el servidor (§12).
- **Salida HTML:** `htmlspecialchars()` sistemático en las vistas al imprimir datos de usuario (previene XSS reflejado/almacenado).
- **Cuentas desactivadas:** `Auth::attempt()` rechaza el login si `usuario.activo = 0`, aunque la contraseña sea correcta.

15. Convenciones de código

- **Idioma:** nombres de tablas, columnas, clases, métodos y comentarios en español (Usuario, listarTrabajadoresPorEmpresa, requireRole...). Mantener esta convención al añadir código nuevo.
- **Un modelo por tabla principal,** con métodos con nombres descriptivos de la consulta que hacen (listarPorEmpresa, buscarValidoPorHash...), sin generalizar en un ORM.
- **Controladores finos:** la lógica de negocio no trivial vive en el modelo o en un método privado del propio controlador (p. ej. guardarImagen() en JuegoController, puedeGestionar() en UsuarioController); los controladores orquestan, no calculan.
- **Vistas sin lógica compleja:** como mucho un foreach/if de presentación; cualquier decisión de negocio (qué puede ver/hacer cada rol) se calcula en el controlador y se pasa ya resuelta, o se consulta con Auth:: directamente en la vista solo para mostrar/ocultar UI (nunca para autorizar).
- **PHP tipado:** parámetros y retornos de métodos con type hints (: array, : ?array, : bool, : void...) y propiedades tipadas (private PDO \$db).
- **Sin dependencias externas:** el proyecto no usa Composer; cualquier funcionalidad (envío de email, etc.) se implementa con las extensiones nativas de PHP. Antes de añadir una librería de terceros, valorar si merece la pena introducir un gestor de dependencias al proyecto.

16. Guías de extensión

- **Añadir una nueva ruta/acción a un controlador existente:** crear el método público en el controlador y registrar la ruta en *public/index.php* (no hay autodescubrimiento de rutas).
- **Añadir un controlador nuevo:** crear *app/controllers/MiController.php* (clase *MiController* extiende *Controller*) y registrar sus rutas — se carga bajo demanda por el Router, no hace falta tocar el *index.php* más allá de las rutas.
- **Añadir un modelo nuevo:** basta con crear *app/models/MiModelo.php*; se incluye automáticamente por el *glob()* de *public/index.php*.
- **Añadir una tabla nueva:** actualizar *database/skillin.sql* (definición +, si aplica, datos de prueba) y aplicar el mismo cambio con *ALTER TABLE* sobre cualquier base de datos ya desplegada (el *.sql* es un script de creación desde cero, no un sistema de migraciones incremental).
- **Añadir un rol o matizar permisos existentes:** el punto de partida es *Controller::requireRole()* (para accesos “todo o nada” a una sección) y *Controller::empresaSeleccionada()* (para alcance por empresa); para reglas más finas de “quién puede tocar qué fila”, seguir el patrón de *UsuarioController::puedeGestionar()*.
- **Añadir más *serious games*.**

17. Limitaciones conocidas y mejoras futuras

- **No hay migraciones:** *skillin.sql* es un script de creación completo (DROP DATABASE + CREATE); cualquier cambio de esquema sobre una base de datos con datos reales requiere un ALTER TABLE manual aparte (no versionado automáticamente).
- **No hay tests automatizados** ni composer.json/herramienta de calidad de código; la verificación se hace manualmente y con php -l para sintaxis.
- **Sin edición de rol de una cuenta ya creada:** el endpoint UsuarioController::actualizar() no toca rol ni id_empresa; cambiar el rol de un usuario existente requeriría ampliar ese formulario (hoy no tiene UI de edición completa, solo alta, activar/desactivar y eliminar).
- **Catálogo de juegos global:** no está pensado para que cada empresa tenga un catálogo distinto; cualquier cambio en ese sentido implicaría añadir id_empresa (o una tabla puente) a juego.
- **Mailer sin reintentos ni cola:** el envío de email es síncrono y de un único intento; un fallo de red durante el registro de una recuperación de contraseña no bloquea la generación del token, pero tampoco reintenta el envío.
- **Sin límite de intentos de login** ni rate limiting explícito (más allá de lo que ofrezca el propio servidor/infraestructura) - quedaría pendiente para un entorno de producción real.



**Plataforma web gamificada para la evaluación y entrenamiento
de competencias profesionales en empresas**